



NANYANG TECHNOLOGICAL UNIVERSITY

EE6407 Genetic Algorithms & Machine Learning

Assignment 1 & 2 Report

MSc in Electrical and Electronic Engineering

Yan Ziming

G2507084J

October 13, 2025

Contents

1	Introduction	2
2	Assignment 1	2
2.1	Handling Missing Values and Outliers	2
2.1.1	Missing values	2
2.1.2	Outliers	3
2.2	Design a Naïve Bayes classifier	4
2.3	Prediction and Analysis	5
3	Assignment 2	6
3.1	Fisher Linear Discriminate Classifier	6
3.2	Prediction and Analysis	6
4	Conclusion	6

1 Introduction

For the course EE6407, I implement two assignments, including the Naïve Bayes Classifier and the Fisher Linear Discriminate Classifier. This report summarizes the methods and results of these two assignments. I use Python as the programming language and Jupyter Notebook as the development environment. The followings show the details of my design and implementation.

2 Assignment 1

2.1 Handling Missing Values and Outliers

2.1.1 Missing values

Detection:

For the missing value detection, I use the pandas library to check, with `isna()` function. This function would return a matrix reflects if the content in each position is empty or not. By summing the values in each column, I can determine the number of times each feature's missing values occur 1.

```
missing_train=Train_data.isna()  
missing_train_n=missing_train.sum()
```

Figure 1: Missing Value Detection Code

At first, there seems no missing values. however, after trying to transform data type from `dtype` to `float`, there seems several data cannot successfully transformed, leading to new missing values. The missing value detection for training set consequence:

Feature	Missing Value Count
A	2
B	0
C	0
D	1
E	0

Handling:

2.1.2 Outliers

Detection:

abcd

Handling:

2.2 Design a Naïve Bayes classifier

```
class GNB_Classifier:
    def __init__(self):
        pass

    def fit(self,X,y):
        self.classes=np.unique(y)
        self.means=[]
        self.vars=[]
        self.priors=[]
        for w in self.classes:
            X_w=X[y==w]
            prior_w=X_w.shape[0]/X.shape[0]
            mean_w=np.mean(X_w,axis=0)

# cov_w=np.cov(X_w,rowvar=False)+1e-6*np.eye
(X.shape[1])
            var_w = np.var(X_w, axis=0)
            self.priors.append(prior_w)
            self.means.append(mean_w)
            # self.vars.append(cov_w)
            self.vars.append(var_w)

        self.priors = np.array(self.priors)
        self.means = np.array(self.means)
        self.vars = np.array(self.vars)

    def ConditionPwithGaussian(self,x,mean,
var):
        eps = 1e-6
        coeff = 1.0 / np.sqrt(2 * np.pi *
var + eps)
        exponent = - (x - mean) ** 2 / 2 *
var + eps)
        return coeff * np.exp(exponent)
```

```

def predict(self, X):
    X = np.array(X)
    y_pred = []

    for x in X:
        posteriors = []
        for i, c in enumerate(self.classes):
            prior_log = np.log(self.priors[i])

            likelihood_log = np.sum(np.log(
self.ConditionPwithGaussian(x, self.means[i]
], self.vars[i])))
            posteriors.append(prior_log +
likelihood_log)
            y_pred.append(self.classes[np.argmax
(posterior)])

        return np.array(y_pred)

def evaluate(self, X, y):
    y = np.array(y)
    y_pred = self.predict(X)
    accuracy = np.mean(y_pred == y)
    return accuracy

```

Figure 2: Code of Classifier Establishment

2.3 Prediction and Analysis

Table 1: Results of Gaussian Naive Bayes Classifier

Metric	Value
Training Accuracy	0.958
Predicted Labels	[1 1 1 1 1 1 1 1 1 3 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3]

3 Assignment 2

3.1 Fisher Linear Discriminate Classifier

Establish:

Training:

3.2 Prediction and Analysis

Table 2: Results of Fisher Linear Discriminant Analysis (LDA)

Metric	Value
Predicted Labels	-1 -1 -1 -1 -1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 1 1 1 1
w	0.001786 0.003473 0.001947 0.001683 0.002658
b	-0.008700
Training Accuracy	0.932

4 Conclusion